

Medii de proiectare și programare

2025-2026

Curs 9

Conținut

- Object-Relational Mapping (ORM)
 - Hibernate
 - Entity Framework
- Servicii Web
- REST

Exemplu

- Etape:
 - Crearea claselor Java corespunzătoare entităților
 - Adăugarea informațiilor pentru mapare (adnotări)
 - Crearea fișierului de configurare Hibernate
 - Implementarea nivelului de persistență folosind funcțiile corespunzătoare
 - Testarea claselor

Exemplu - Entitatea

@Entity

```
public class Book {  
    private Long id;  
    private String title;  
  
    public Book() {} //obligatoriu  
    public Book(String title){ this.title=title;}  
  
    @Id  
    @GeneratedValue(strategy=IDENTITY)  
    public Long getId() { return id;}  
  
    public void setId(Long id) {this.id = id;}  
  
    @NotNull  
    public String getTitle() { return title;}  
  
    public void setTitle(String title) {this.title = title;}  
}
```

Exemplu - Entitatea

```
@Entity
@Table( name = "Messages")
public class Message {
    @Id
    @Column(name = "idMessage")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "Message_Text")
    private String text;

    public Message() {} //obligatoriu
    public Message(String text) { this.text = text; }

    public Long getId() { return id; }
    private void setId(Long id) { this.id = id; }

    public String getText() { return text; }
    public void setText(String text) { this.text = text; }
}
```

Fișierul de configurare *hibernate.properties*

```
#hibernate.properties –numele implicit căutat de Hibernate
```

```
# URL-ul de conectare la baza de date (JDBC)
```

```
jakarta.persistence.jdbc.url=jdbc:sqlite:/Users/grigo/HibernateExample/hibernate6.db
```

```
# Credentials
```

```
#jakarta.persistence.jdbc.user=
```

```
#jakarta.persistence.jdbc.password=
```

```
jakarta.persistence.schema-generation.database.action=update
```

```
#none/create/etc
```

```
# SQL statement logging
```

```
hibernate.show_sql=true
```

```
hibernate.format_sql=true
```

```
hibernate.highlight_sql=true
```

Exemplu – Salvarea unei entități

```
//INSERT
```

```
void addBook(Book b) {  
    sessionFactory.inTransaction(session -> {  
        session.persist(b);  
        System.out.println("Book added "+b);  
    });  
}
```

```
//Book b=new Book("Test Book ");
```

```
//dupa rulare
```

```
//Book added Book{id='8', title='Test Book '}
```


Exemplu – Modificarea unei entități

```
//FindOne
Book findBook(int id){
    try(Session session=sessionFactory.openSession()){
        return session.find(Book.class, id);
    } //se apelează session.close()
    catch (Exception e){
        System.err.println("Eroare la findBook " +e);
    }
    return null;
}

//update
Book update(int id){
    Book b=findBook(id);
    b.setTitle("New Title");
    sessionFactory.inTransaction(session->{
        session.merge(b);
        //session.flush()
    });
    return b;
}
```


Exemplu – ștergerea unei entități

```
//DELETE  
void delete(Book book) {  
    sessionFactory.inTransaction(session -> {  
        session.remove(book) ;  
    }) ;  
}
```

Exemplu – selectarea unei entități

```
//SELECT
```

```
List<Book> getBooks() {  
    try( Session session= sessionFactory.openSession()) {  
        return session.createQuery("from Book where title like 'Test %'",  
Book.class).setFirstResult(0).  
            setMaxResults(5).list();  
    } //se apelează session.close()  
}
```

```
//Book - numele entității, NU a tabelii
```

```
//title - proprietate a clasei Book, NU numele coloanei din tabela
```

Exemplu – MainBook

```
public class MainBook {
    static SessionFactory sessionFactory;
    public static void main(String[] args) {
        // A SessionFactory is set up once for an application!
        sessionFactory = new Configuration() //cauta fisierul hibernate.properties
            .addAnnotatedClass(Book.class)
            .buildSessionFactory();

        Book b=new Book("Test Book ");
        // persist an entity
        addBook(b);

        System.out.println("Searching book having id 1 "+findBook(1));
        update(6);

        // query data using HQL
        for(Book bb:getBooks())
            System.out.println(bb);

        //delete an entity
        delete(b);

        //close sessionFactory
        sessionFactory.close();
    }
    //... codul metodelor addBook, findBook, update, getBooks, delete
}
```

Configurare Gradle

```
plugins {  
    id 'java'  
    id 'application'  
}  
  
group = 'ro.mpp'  
version = '1.0'  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
  
    implementation 'org.hibernate.orm:hibernate-core:6.4.4.Final'  
  
    // Hibernate Validator  
    implementation 'org.hibernate.validator:hibernate-validator:8.0.0.Final'  
    implementation 'org.glassfish:jakarta.el:4.0.2'  
  
    //pentru lucrul cu diferite baze de date: Sqlite, MariaDb, etc...  
    implementation 'org.hibernate.orm:hibernate-community-dialects:6.4.4.Final'  
  
    testImplementation platform('org.junit:junit-bom:5.9.1')  
    testImplementation 'org.junit.jupiter:junit-jupiter'  
  
    // driverul pentru conectarea la o baza de date Sqlite  
    runtimeOnly 'org.xerial:sqlite-jdbc:3.45.3.0'  
}
```

Interogări ale bazei de date

- Două posibilități:

- Hibernate Query Language

```
session.createQuery("from Category c where c.name like  
    'Laptop%'");
```

- SQL

```
session.createNativeQuery(  
    "select {c.*} from CATEGORY {c} where NAME like 'Laptop%'",  
    "c", Category.class);
```

Obținerea rezultatelor

- Metodele `list()/getResultList()` execută interogarea și returnează rezultatul ca și o listă:

```
List<User> result = session.createQuery("from User",  
    User.class).getResultList();
```

- Un singur obiect ca și rezultat :

```
Bid maxBid =session.createQuery("from Bid b order by  
    b.amount desc", Bid.class)  
    .setMaxResults(1)  
    .uniqueResult();
```

Interogări cu parametri

- Parametrii cu nume

```
String queryString = "from Item item where item.description  
    like :searchString and item.date > :minDate";  
List result = session.createQuery(queryString)  
    .setParameter("searchString", searchS)  
    .setParameter("minDate", minD).list();
```

- Parametrii cu poziție:

```
String queryString = "from Item item where item.description  
    like ?1 and item.date > ?2";  
List result = session.createQuery(queryString)  
    .setParameter(1, searchString)  
    .setParameter(2, minDate)  
    .list();
```


Hibernate Query Language (HQL)

Suportă aproape toate funcțiile și operațiile SQL:

- from clause: `from Cat as cat`
- select clause: `select foo from Foo foo, Bar bar where foo.startDate = bar.date`
- where clause: `from Cat as cat where cat.name='Fritz'`
- aggregate functions:

```
select cat.color, sum(cat.weight), count(cat) from Cat cat
group by cat.color
```

- order by clause
- group by clause
- expressions
- etc.

Maparea relațiilor

Adnotări pentru maparea relațiilor dintre clase:

- **@ManyToMany**
- **@ManyToOne**
- **@OneToOne**
- **@JoinTable** - tabela de asociere
- **@JoinColumn**

Fetching - **lazy**, **eager**

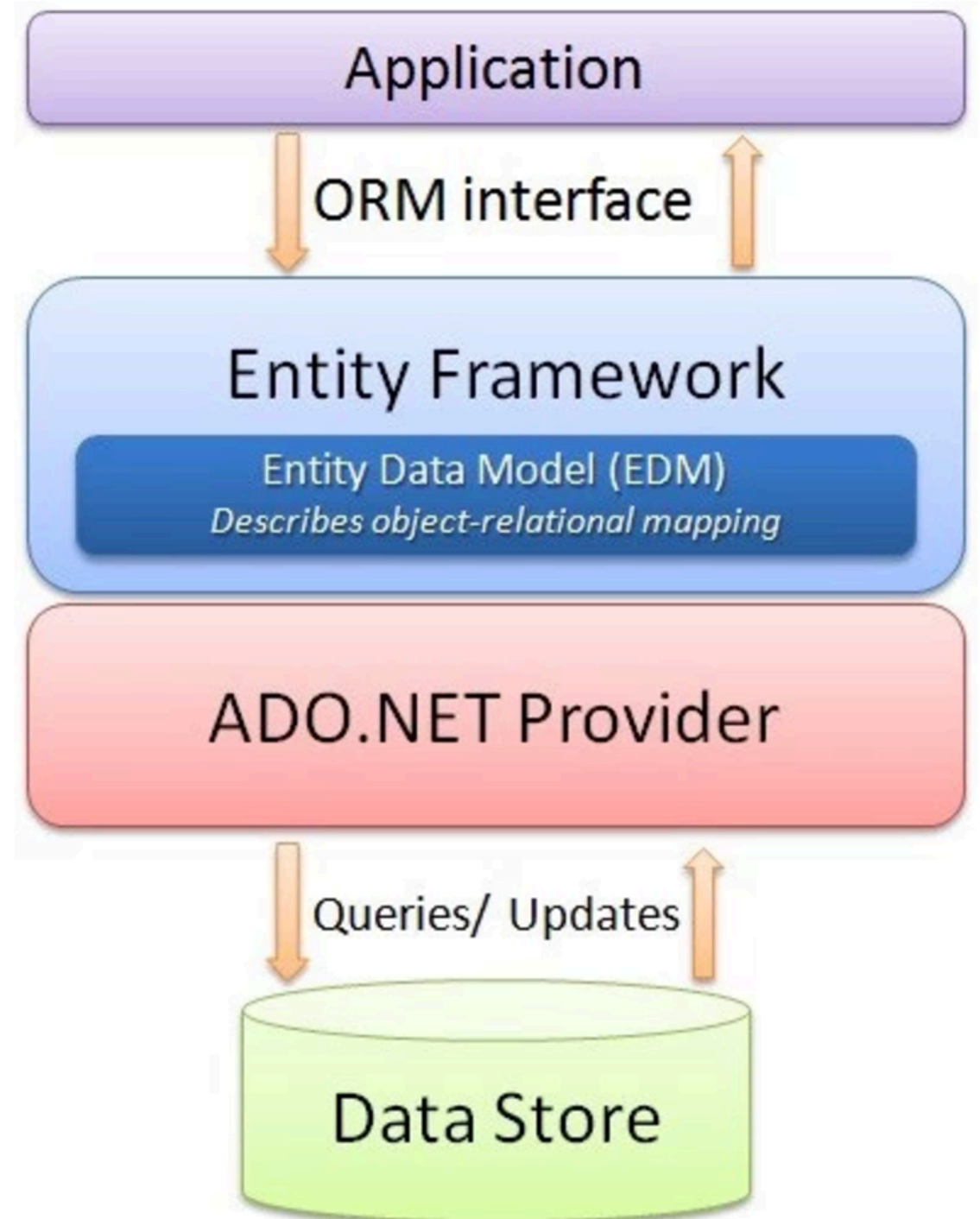
https://docs.jboss.org/hibernate/orm/6.3/introduction/html_single/Hibernate_Introduction.html

- **@MappedSuperclass** (pentru moștenire)

Exemplu Hibernate

.NET ORM

- Instrumente ORM bazate pe LINQ:
 - [LINQ to SQL](#) (L2S)
 - [Entity Framework](#) (EF)
- EF permite o mai bună decuplare a claselor de modelul relațional



.NET L2S

- Decorarea claselor folosind attribute .NET
- Spațiul de nume `System.Data.Linq.Mapping`

```
[Table (Name="Customers")]
public class Customer
{
    [Column(IsPrimaryKey=true)]
    public int ID {get;set};
    [Column (Name="FullName")]
    public string Name {get; set};
}
```

.NET L2S

```
var context = new DataContext ("database connection string");

Table<Customer> customers = context.GetTable <Customer>();

// numărul de înregistrări din tabelă.
Console.WriteLine (customers.Count());

// Clientul cu Id-ul 2.
Customer cust = customers.Single (c => c.ID == 2);

Customer cust = customers.OrderBy (c => c.Name).First();
cust.Name = "Updated Name";
context.SubmitChanges();
```

.NET EF

- Decorarea claselor folosind attribute .NET
- Referință către [System.Data.Entity.dll](#)

```
[EdmEntityType (NamespaceName = "EFModel", Name = "Customer")]  
public partial class Customer  
{  
    [EdmScalarPropertyAttribute (EntityKeyProperty=true,  
        IsNullable=false)]  
    public int ID { get; set; }  
  
    [EdmScalarProperty (EntityKeyProperty = false, IsNullable = false)]  
        public string Name { get; set; }  
}
```

.NET EF

```
var context = new ObjectContext ("entity connection string");
context.DefaultContainerName = "EntitiesContainer";
ObjectSet<Customer> customers =
context.CreateObjectSet<Customer>();

// numărul de înregistrări din tabelă
Console.WriteLine (customers.Count());

// Clientul cu ID-ul 2
Customer cust = customers.Single (c => c.ID == 2);

Customer cust = customers.OrderBy (c => c.Name).First();
cust.Name = "Updated Name";
context.SaveChanges();
```


Servicii Web & REST

Servicii Web

- Definiții:

1. Un **serviciu web** este o aplicație/componentă soft disponibilă pe internet și care folosește un sistem standardizat de transmitere a mesajelor în format XML. XML este folosit pentru comunicarea datelor către/de la un serviciu web.
 - Un client apelează un serviciu web trimițând un mesaj în format XML și așteaptă un răspuns în format XML.
 - Comunicarea are loc folosind XML, serviciile web nu sunt dependente de un anumit sistem de operare sau de un anumit limbaj de programare.
2. **Serviciile web** sunt aplicații *self-contained*, modulare și distribuite care pot fi descrise, publicate, localizate și apelate prin rețea pentru a crea produse, procese, etc. Aceste aplicații pot fi locale, distribuite sau pe web. Serviciile web sunt dezvoltate folosind standarde ca TCP/IP, HTTP și XML.
3. **Serviciile web** sunt sisteme de schimbare a mesajelor bazate pe XML care folosesc Internetul pentru interacțiunea dintre aplicații. Aceste sisteme pot fi: programe, obiecte, mesaje, documente, etc.
4. Un **serviciu web** este o colecție de protocoale și standarde folosite pentru transmiterea datelor între aplicații sau sisteme. Aplicații dezvoltate în diferite limbaje de programare și rulând pe diferite platforme (sisteme de operare) pot folosi serviciile web pentru a schimba date în rețea (ex. Internet) într-o manieră similară comunicării între procese pe același calculator. Interoperabilitatea acestora este posibilă datorită folosirii standardelor.

Servicii Web

- Un **serviciu web** este un **serviciu**:
 - Disponibil pe Internet sau într-o rețea privată.
 - Folosește un sistem standardizat de schimbare a mesajelor bazat pe XML.
 - Nu este dependent de un anumit limbaj de programare sau de un anumit sistem de operare.
 - Este self-describing folosind o gramatică XML (DTD, XML Schema).
 - Poate fi descoperit folosind mecanisme simple.
- **Componentele**: structura de baza pentru serviciile web este XML + HTTP. Toate serviciile web standard funcționează folosind componentele:
 - **XML** pentru marcarea informației
 - **SOAP** (Simple Object Access Protocol) pentru transmiterea mesajelor
 - **UDDI** (Universal Description, Discovery and Integration)
 - **WSDL** (Web Services Description Language) pentru a descrie disponibilitatea serviciului

Exemplu WSDL

```
<definitions name = "HelloService"
  targetNamespace = "http://www.examples.com/wsdl/HelloService.wsdl"
  xmlns = "http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap = "http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns = "http://www.examples.com/wsdl/HelloService.wsdl"
  xmlns:xsd = "http://www.w3.org/2001/XMLSchema">

  <message name = "SayHelloRequest">
    <part name = "firstName" type = "xsd:string"/>
  </message>

  <message name = "SayHelloResponse">
    <part name = "greeting" type = "xsd:string"/>
  </message>

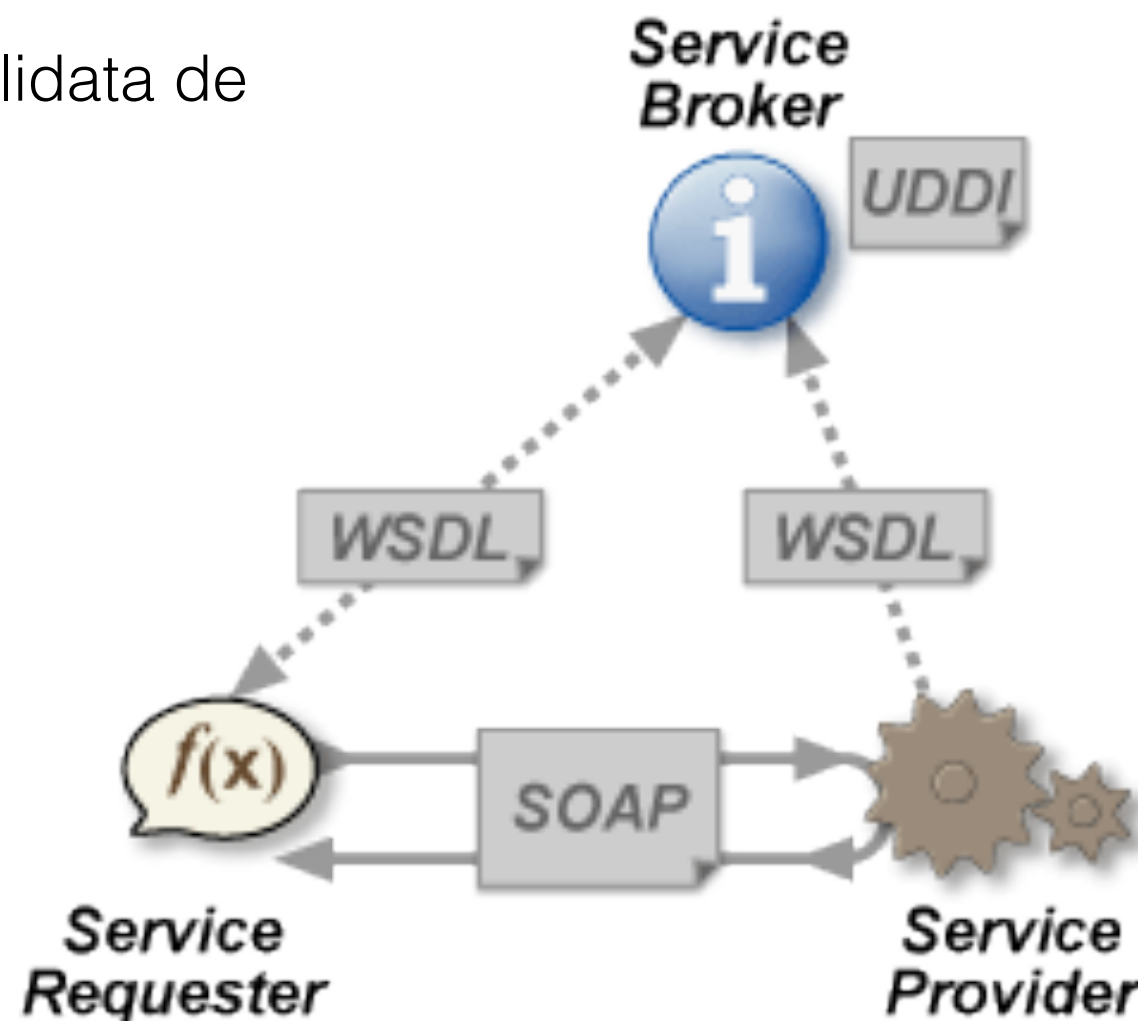
  <portType name = "Hello_PortType">
    <operation name = "sayHello">
      <input message = "tns:SayHelloRequest"/>
      <output message = "tns:SayHelloResponse"/>
    </operation>
  </portType>

  <binding name = "Hello_Binding" type = "tns:Hello_PortType">
    <soap:binding style = "rpc"
      transport = "http://schemas.xmlsoap.org/soap/http"/>
    <operation name = "sayHello">
      <soap:operation soapAction = "sayHello"/>
      <input>
        <soap:body
          encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/"
          namespace = "urn:examples:helloservice"
          use = "encoded"/>
      </input>
      <output>
        <soap:body
          encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/"
          namespace = "urn:examples:helloservice"
          use = "encoded"/>
      </output>
    </operation>
  </binding>

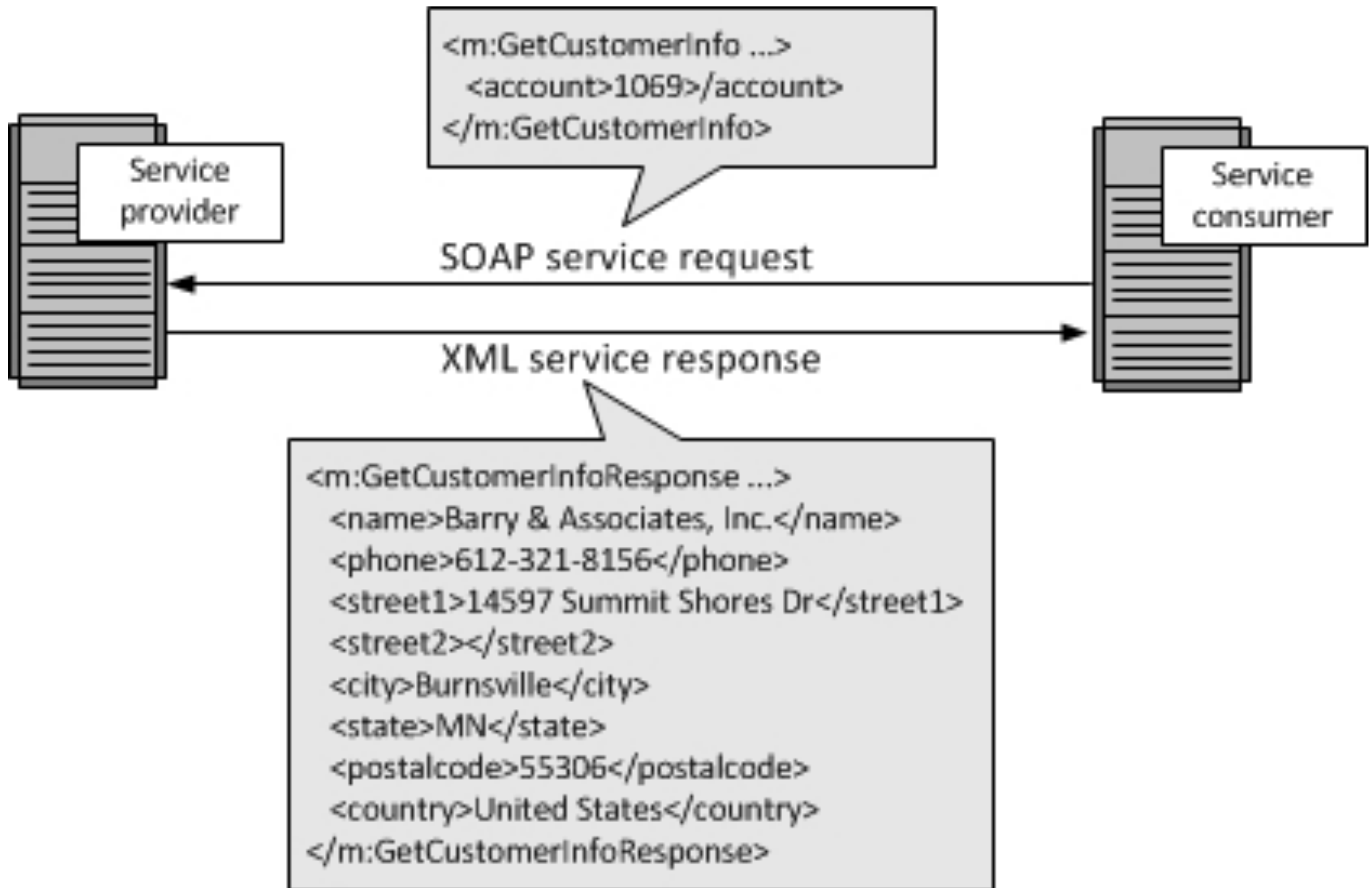
  <service name = "Hello_Service">
    <documentation>WSDL File for HelloService</documentation>
    <port binding = "tns:Hello_Binding" name = "Hello_Port">
      <soap:address
        location = "http://www.examples.com/SayHello/" />
    </port>
  </service>
</definitions>
```

Servicii Web - Arhitectura

- Furnizorul serviciului trimite un fișier WSDL serviciului UDDI.
- Clientul serviciului (consumatorul) contactează UDDI pentru a descoperi cine este furnizorul datelor de care are nevoie, iar apoi contactează furnizorul serviciului folosind protocolul SOAP.
- Furnizorul serviciului validează cererea și trimite datele cerute în format XML folosind protocolul SOAP.
- Structura datelor în format XML ar trebui validată de client folosind un fișier XSD (XML Schema).



Servicii Web - Arhitectura



REST

- REpresentational State Transfer (REST).
- Este un stil arhitectural pentru sisteme distribuite hipermedia.
- A fost propus de Roy T. Fielding în teza sa de doctorat “*Architectural Styles and the Design of Network-based Software Architectures*” (2000).
- Conține 6 constrângeri ce trebuie satisfăcute pentru a fi numit sistem REST veritabil:
 1. Client–server
 2. Fără stare (eng. *stateless*)
 3. Cacheable
 4. Interfață uniformă (eng. *uniform interface*)
 5. Sistem stratificat (eng. *layered system*)
 6. Cod la cerere (eng. *code on demand*) -opțională

REST - Resursa

- Conceptul de bază în REST este *resursa* (eng. *resource*).
- Orice informație care poate fi numită poate fi **resursă**: un document, o imagine, o colecție de alte resurse, un obiect real (ex. persoană, mașină), etc.
- REST folosește *identificatorul resursei* pentru a identifica resursa în momentul interacțiunii între componente.
- Starea unei resurse la un anumit moment de timp este numită **reprezentarea resursei**.
- Reprezentarea constă din *date*, *metadata* care descriu datele și *legături hipermedia* care ajută clienții în tranziția la următoarea stare dorită.
- Formatul datelor dintr-o reprezentare este numit **tip media** (eng. *media type*).
- Tipul media identifică o specificație care definește modul în care reprezentarea va fi procesată.
- Un API RESTful veritabil seamănă cu hypertext. Fiecare informație ce poate fi obținută are o *adresă explicită* (prin legături și attribute id) sau *implicită* (obținută din definiția tipului media și structura reprezentării).
- Reprezentările resurselor trebuie să se descrie pe ele (eng. *self-descriptive*): clientul nu trebuie să știe că resursa este persoană sau mașină. Datele trebuie să poată fi procesate pe baza tipului media asociat resursei.

REST - metode asupra resursei

- Un lucru important asociat cu REST sunt metodele/operațiile ce pot fi aplicate resursei pentru a efectua tranziția dorită.
- Toată informația necesară modificării stării resursei face parte din API-ul cererii pentru acea resursă (inclusiv operațiile/metodele și modul în care vor afecta reprezentarea resursei).
- Este recomandat ca rezultatele interogărilor să fie reprezentate folosind o listă cu legături conținând informații sumare, și nu tablouri conținând toate informațiile corespunzătoare resursei, deoarece interogarea nu este un substitut pentru identificarea resurselor.
- Adesea, metodele/operațiile asupra resursei sunt confundate cu metodele HTTP GET/PUT/POST/DELETE.

REST și HTTP

- REST și HTTP nu reprezintă același lucru.
- Dacă un sistem satisface constrângerile REST, se poate spune ca sistemul este RESTful.
- În stilul arhitectural REST, datele și funcționalitățile sunt considerate resurse ce pot fi accesate folosind *Uniform Resource Identifiers* (URIs).
- Resursele sunt prelucrate (adăugate, șterse, modificate, etc) folosind un set simplu, bine definit de operații.
- Clienții și serverele interschimbă (schimbă între ele) reprezentări ale resurselor folosind o interfață standardizată și un protocol standardizat (adesea HTTP).
- Resursele sunt decuplate de reprezentarea lor. Conținutul lor poate fi accesat folosind diferite formate: XML, text, PDF, JSON, HTML, etc.
- Metadata despre resurse este folosită pentru detecția erorilor, controlul caching-ului, negocierea celui mai convenabil format pentru reprezentare, autentificare și controlul accesului.
- Fiecare interacțiune este stateless.
- Aceste principii fac aplicațiile RESTful simple, lightweight și rapide.

Constrângerile arhitecturale REST

- REST este un stil arhitectural ce permite proiectarea unor aplicații slab cuplate (loosely coupled) folosind HTTP.
- Este adesea folosit pentru dezvoltarea serviciilor web.
- REST nu impune nici o regulă despre cum ar trebui implementat la nivelele inferioare, doar impune constrângeri la nivelul de proiectare și lasă implementarea dezvoltatorului.
- REST definește 6 constrângeri arhitecturale a căror satisfacere fac orice serviciu web un API RESTful veritabil:
 1. *Uniform interface*
 2. *Client-server*
 3. *Stateless*
 4. *Cacheable*
 5. *Layered system*
 6. *Code on demand* (opțional)

REST - *Uniform interface*

- Aplicând principiul generalității interfeței componentelor care interacționează, arhitectura întregului sistem este simplificată și interacțiunea între componente este îmbunătățită.
- Pentru a obține o interfață uniformă, se folosesc mai multe constrângeri arhitecturale pentru a ghida execuția componentelor.
- REST este definit de 4 constrângeri ale interfeței:
 - *Identificarea resurselor*
 - *Manipularea resurselor folosind reprezentările*
 - *Mesaje ce se descriu pe sine (self-descriptive)*
 - *Hypermedia ca și motor al stării aplicației.*
- Dezvoltatorii trebuie să decidă interfața pentru resursele sistemului făcute publice clienților și să **folosească întotdeauna această interfață**.
- O resursă din sistem trebuie să aibă un singur URI, și trebuie să ofere o modalitate de a obține date adiționale sau asociate.
- O resursă nu ar trebui să fie foarte mare și ar trebui să conțină legături către alte informații adiționale (URI relativ) care permit obținerea acelor informații.
- Reprezentările resurselor trebuie să respecte anumite recomandări (convenții de nume, formatul legăturilor, formatul datelor -xml și/sau json).
- Toate resursele ar trebui să fie accesibile folosind o abordare comună (ex. folosind HTTP GET) și, în mod similar, ar trebui să poată fi modificate folosind o abordare consistentă.

REST - *Client-server*

- Aplicațiile client și server trebuie să poată evolua independent fără dependențe între ele.
- Clientul ar trebui să știe doar de URI-ul resursei.
- Prin separarea interfeței clientului de modul de păstrare a resurselor, se îmbunătățește portabilitatea interfeței cu utilizatorul pe platforme diferite și se îmbunătățește scalabilitatea prin simplificarea componentelor de pe server.
- “*Servers and clients may also be replaced and developed independently, as long as the interface between them is not altered.*”

REST - *Stateless*

- Stilul REST a fost inspirat din HTTP.
- Toate interacțiunile client-server trebuie să fie **stateless**.
- Serverul nu va păstra informații despre ultima cerere HTTP de la client.
- Fiecare cerere va fi tratată ca și cum ar fi o cerere nouă (fără sesiune, fără istoric).
- Fiecare cerere trimisă de la client la server trebuie să conțină toate informațiile necesare pentru a înțelege cererea și nu poate folosi informații legate de context păstrate pe server.
- Starea sesiunii este păstrată în întregime la client.
- Dacă o aplicație trebuie să păstreze starea pentru client (clientul se autentifică iar apoi efectuează operații), fiecare cerere de la client trebuie să conțină toate datele necesare pentru îndeplinirea cererii (inclusiv detalii legate de autentificare și autorizare).
- “*No client context shall be stored on the server between requests. Client is responsible for managing the state of application.*”

Constrângeri REST

- *Cacheable*: constrângerea cere ca datele dintr-un răspuns la o cerere să fie **implicit sau explicit marcate ca cacheable sau non-cacheable**. Dacă răspunsul este cacheable, atunci cache-ul de pe client are permisiunea de a refolosi datele ulterior (echivalând cu obținerea lui folosind cereri către server).
- Caching îmbunătățește performanța clientului și îmbunătățește scalabilitatea serverului deoarece numărul de cereri se reduce.
- Caching trebuie aplicat tuturor resurselor ce se declară cacheable. Caching poate fi implementat atât pe server cât și pe client.
- “*Well-managed caching partially or completely eliminates some client–server interactions, further improving scalability and performance.*”
- *Sistem stratificat*:
 - REST permite folosirea unei arhitecturi stratificate a sistemului: API public (URIs) este pe serverul A, datele sunt păstrate pe server B, iar cererile de autentificare se fac pe serverul C.
 - Un client nu știe dacă este conectat direct cu serverul destinație sau cu un intermediar.
- Arhitectura stratificată permite ca sistemul final să fie compus din nivele ierarhice care permit unui nivel comunicarea doar cu nivelul inferior (superior)

Constrângeri REST

- *Code on demand*
- Constrângerea este **opțională**. Adesea, serverele vor trimite reprezentarea statică a resurselor în format XML sau JSON.
- *Serverul poate trimite și cod executabil, pentru a suporta o parte a aplicației* (ex. cod pentru afișarea unei componente grafice)
- REST permite extinderea funcționalității clienților prin descărcarea și executarea codului (sub forma unor applet-uri sau a unor scripturi).
- *Poate simplifica clienții prin reducerea numărului de funcționalități ce trebuie implementate ulterior.*

Recomandări resurse REST

- Se recomandă folosirea consecventă a unei strategii pentru denumirea resurselor.
- API REST folosește URIs pentru accesarea resurselor. URI-urile folosite ar trebui să sugereze clienților modelul resurselor.
- Dacă resursele sunt denumite sugestiv, API-ul corespunzător va fi ușor de folosit și intuitiv. Altfel poate fi dificil de înțeles și utilizat.
- O resursă poate fi *singulară* sau *o colecție de alte resurse*.

Exemplu:

- “**articles**” - colecție de resurse
- “**article**” - singular.
- Identificarea resurselor de tip colecție se poate face folosind substantivul la plural:
Resursa “**articles**” : URI “/**articles**”.
- Identificarea unei resurse singulare se poate face folosind identificatorul:
Resursa “**article**” are URI “/**articles**/**{articleID}**”.
- O resursă poate conține o colecție de alte resurse (subcolecție).
ex. autorii unui articol pot fi identificați prin:
“/**articles**/**{articleID}**/**authors**”.
- Identificarea unei resurse singulare dintr-o subcolecție:
“/**articles**/**{articleID}**/**authors**/**{authorID}**”.

REST Recomandări

- *Folosirea substantivelor pentru reprezentarea resurselor*
- **URI RESTful trebuie să refere resursa, nu să se refere la o acțiune.** Substantivele au proprietăți, și asemănător resursele au attribute.

`http://api.example.com/conference/articles`

~~`http://api.example.com/conference/getArticles`~~ - nu este URI corespunzator REST

`http://api.example.com/conference/articles/{id}/title`

`http://api.example.com/user-management/users`

`http://api.example.com/user-management/users/{id}`

`http://api.example.com/user-management/users/{id}/name`

- *Folosirea consecventă a convențiilor pentru denumirea resurselor și a modului de formare a URI-ului pentru a reduce ambiguitățile și îmbunătățirea înțelegerii și a întreținerii:*

- Folosirea caracterului / pentru a indica relații ierarhice
- A NU se folosi caracterul / la finalul URI-ului

~~`http://api.example.com/device-management/managed-devices/`~~

`http://api.example.com/device-management/managed-devices`

REST Recomandări

- *Folosirea cratimei (-) pentru îmbunătățirea înțelegerii URI*

<http://api.example.com/inventory-management/managed-entities/{id}/install-script-location>

~~<http://api.example.com/inventory-management/managedEntities/{id}/installScriptLocation>~~

- *Folosirea literelor mici în URI*

<http://api.example.org/my-folder/my-doc>

~~<http://api.example.org/My-Folder/my-doc>~~

- *A nu se folosi extensii de fișier:* Nu aduc informații suplimentare clientului și pot induce în eroare dezvoltatorii.
- Pentru a preciza *tipul media corespunzător unei reprezentări* se folosesc antetele *Accept* și *Content-Type*.
- Acestea sunt folosite și pentru a determina modul de procesare a conținutului.

~~<http://api.example.com/device-management/managed-devices.xml>~~

<http://api.example.com/device-management/managed-devices>

REST Recomandări

- *A nu se folosi nume de funcții CRUD (get, add/save, delete, update/modify) în URI-uri*
- URI-urile nu ar trebui folosite pentru a preciza operația/operațiile efectuate.
- URI-urile ar trebui folosite pentru a identifica în mod unic resursele, nu pentru a le manipula.
- Metodele HTTP ar trebui folosite pentru a indica operația CRUD ce trebuie efectuată.

//obținerea tuturor articolelor (echivalentul lui getAll/findAll)

HTTP GET si URI <http://api.example.com/conference/articles>

//Crearea unui nou articol (echivalentul lui save/add)

HTTP POST si URI <http://api.example.com/conference/articles>

//Obținerea articolului cu id-ul dat (echivalentul lui findOne)

HTTP GET si URI <http://api.example.com/conference/articles/{id}>

//Actualizarea articolului cu id-ul dat (echivalentul lui update/modify)

HTTP PUT si URI <http://api.example.com/conference/articles/{id}>

//Stergerea articolului cu id-ul dat (echivalentul lui delete)

HTTP DELETE si URI <http://api.example.com/conference/articles/{id}>

REST Recomandări

- *Folosirea parametrilor interogării HTTP pentru a filtra o colecție.*
 - Sortarea elementelor din colecție
 - Filtrarea elementelor din colecție
 - Limitarea numărului de elemente returnate (paginarea)
 - Nu se creează URI-uri noi, ci se folosesc parametrii de tip query:

`http://api.example.com/conference/articles`

`http://api.example.com/conference/articles?domain=AI`

`http://api.example.com/conference/articles?domain=AI&subdomain=Genetic`

`http://api.example.com/conference/articles?`

`domain=AI&subdomain=Genetic&sort=submission-date`

`//gresit`

`http://api.example.com/conference/getArticlesSortedBy?`

`domain=AI&subdomain=Genetic&sort=submission-date`

REST API - Proiectare

- *Pașii proiectării serviciilor REST*

1. Identificarea modelului obiectual
2. Crearea modelului pentru URI-uri
3. Determinarea reprezentărilor
4. Atribuirea/asocierea metodelor HTTP
5. Alte acțiuni (Logging, Security, Discovery)

- Identificarea modelului obiectual

- Identificarea obiectelor care vor fi prezentate ca și resurse:
 - Users
 - Messages,
 - Articles,
 - Authors,
 - Persons
 - etc.

REST API Proiectare

- Crearea modelului URI-urilor
 - Deciderea URI-urilor pentru resursele identificate anterior.
 - Axarea pe relațiile dintre resurse și sub-resurse.
 - Aceste URI-uri asociate resurselor vor fi folosite ca și endpoint-uri pentru serviciile REST.
 - **URI-urile nu conțin verbe sau operații.**
 - **URI-urile ar trebui să conțină doar substantive.**

`/articles`

`/articles/{id}`

`/authors`

`/authors/{id}`

`/articles/{id}/authors`

`/articles/{id}/authors/{aid}`

REST API - Proiectare

- *Determinarea reprezentării resurselor*

- Majoritatea reprezentărilor sunt definite în format XML sau JSON.

<code><author></code>	<code>{</code>
<code> <name> Pop Ion </name></code>	<code> "name": "Pop Ion"</code>
<code> <affiliation> ZY Lab </affiliation></code>	<code> "affiliation": "ZY Lab"</code>
<code></author></code>	<code>}</code>

- *Asocierea metodelor HTTP*

- Obținerea tuturor articolelor sau autorilor

HTTP GET si URI **/articles**

HTTP GET si URI **/authors**

- Paginarea, dacă rezultatul este prea mare.

HTTP GET si URI **/articles?startIndex=0&size=20**

HTTP GET si URI **/authors?startIndex=0&size=20**

- Obținerea tuturor autorilor unui articol (subcolecție)

HTTP GET si URI **/articles/{id}/authors**

REST API -Proiectare

- Obținerea unui singur articol/autor

HTTP GET si URI **/articles/{id}**

HTTP GET si URI **/authors/{id}**

- Obținerea unui singur autor (subcolecție)

HTTP GET si URI **/articles/{id}/authors/{id}**

Reprezentarea subresursei poate fi diferită.

- Crearea unui articol/autor - metoda HTTP POST

HTTP POST si URI **/articles**

HTTP POST si URI **/authors**

IMPORTANT!

- **De obicei, la crearea unei resurse, corpul cererii NU conține informații legate de id, deoarece serverul este responsabil de determinarea acestuia.**
- **Exceptie:** id-ul este informatie relevanta pentru resursa (username, CNP)

REST API - Proiectare

- Actualizarea unui articol/autor - metoda HTTP PUT

HTTP PUT si URI **/articles/{id}**

HTTP PUT si URI **/authors/{id}**

- Ștergerea unui articol/autor - metoda HTTP DELETE

HTTP DELETE si URI **/articles/{id}**

HTTP DELETE si URI **/authors/{id}**

- Un răspuns ar trebui să returneze:

- **202 (Accepted)** dacă resursa a fost pusă într-o coadă și urmează a fi ștearsă (asincron).
- **200 (OK) / 204 (No Content)** dacă resursă a fost ștearsă permanent (sincron).

- Adăugarea unui autor la articol - Metoda HTTP PUT

HTTP PUT si URI **/articles/{id}/authors**

- Ștergerea unui autor de la articol - Metoda HTTP DELETE

HTTP DELETE si URI **/articles/{articleId}/authors/{authorId}**

HTTP Response Status Code

- **2XX -Success** acțiunea cerută de client a fost primită, înțeleasă, acceptată și procesată cu succes.
 - **200 OK**: Răspunsul standard pentru cereri HTTP executate cu succes.
 - **201 Created**: Cererea a fost îndeplinită, și a fost creată o nouă resursă.
 - **202 Accepted**: Cererea a fost acceptată pentru procesare, dar procesarea nu a fost încă efectuată.
 - **204 No Content**: Serverul a procesat cererea cu succes, dar procesarea nu a returnat date.
 - **205 Reset Content**: Serverul a procesat cererea cu succes, dar nu a returnat date. Spre deosebire de răspunsul 204, acest răspuns necesită ca clientul să reafișeze vederea/informatia.
- etc.

HTTP Response Status Code

- **4XX** Erori client:
 - **400 Bad Request**: Serverul nu poate sau nu va procesa cererea din cauza unei erori din partea clientului (ex. sintaxa cererii este greșită, conținutul cererii este prea mare, etc.)
 - **401 Unauthorized**: Similar cu 403, dar se folosește când este necesară autentificarea, care încă nu s-a efectuat sau a eșuat.
 - **403 Forbidden**: Cererea este validă, dar serverul refuză procesarea. (Utilizatorul nu are drepturile necesare pentru a accesa resursa respectivă).
 - **404 Not Found**: Resursa ceruta nu a fost găsită, dar poate deveni disponibilă în viitor. Sunt permise cereri ulterioare din partea clientului.
 - **405 Method Not Allowed**: Metoda cerută nu este disponibilă pentru resursa cerută. Ex. o cerere GET pentru o formă care necesită ca datele să fie transmise folosind POST, sau o cerere PUT pentru o resursă care poate fi doar citită.
 - **406 Not Acceptable**: Reprezentările disponibile pentru resursa ceruta nu sunt acceptate de client (folosind antetul *Accept*).
 - etc.
- **5XX** Erori la implementarea serviciilor

HTTP Media Types

- Antetele HTTP **Accept** și **Content-Type** pot fi folosite pentru a descrie reprezentarea folosită pentru schimbarea informației (reprezentarea resurselor).
- Clientul trimite o cerere în care în antetul **Accept** specifică lista tipurilor de reprezentare acceptate (separate prin virgulă).

Exemplu : **Accept: application/json, application/xml**

- Serverul trimite răspunsul folosind ca și reprezentare primul tip mime înțeles din lista precizată (dacă înțelege unul sau mai multe). Tipul folosit este precizat în antetul **Content-Type**.

Exemplu: **Content-Type: application/xml**

Referințe

- Roy Fielding, PhD Thesis, Chapter 5. Representational State Transfer (REST)

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

- <http://restfulapi.net/>
- <http://www.service-architecture.com/articles/web-services/>
- Alte tutoriale